ingenieur
wissenschaften
htw saar

Hochschule für
Technik und Wirtschaft
des Saarlandes
University of
Applied Sciences

Audioverarbeitung mittels eines Computers

von

Amsakan Bavan

Ein wissenschaftlicher Bericht im Rahmen der Vorlesung

„Wissenschaftliches Arbeiten“

an der htw saar im Studiengang Informatik

Saarbrücken, den 08.07.2025

Abstract

In dieser Arbeit befassen wir uns mit der Audioverarbeitung von Computern. Dabei beleuchten wir zunächst einige theoretische Grundlagen, welche auf der digitalen Signalverarbeitung basieren und setzen diese durch Implementierung um, sodass ein tieferes Verständnis bezüglich der Audioverarbeitung auf Computern aufgebaut wird.

Selbstständigkeitserklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Insbesondere habe ich alle KI-basierten Werkzeuge angegeben, die ich bei der Erstellung, Übersetzung oder Überarbeitung des Textes verwendet habe.

Ich erkläre hiermit weiterhin, dass die vorgelegte Arbeit zuvor weder von mir noch von einer anderen Person an dieser oder einer anderen Hochschule eingereicht wurde.

Darüber hinaus ist mir bekannt, dass die Unrichtigkeit dieser Erklärung eine Benotung der Arbeit mit der Note „nicht ausreichend“ zur Folge hat und einen Ausschluss von der Erbringung weiterer Prüfungsleistungen zur Folge haben kann.

Saarbrücken, den 01.01.1970

Unterschrift

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Grundlagen und Begriffe 1

2 Bestehende Lösungen **1**
2.1 Bestehe Lösungen zur Audiokomprimierung 2
2.2 Bestehende Lösungen zur Audiobearbeitung 2

3 Theorie **2**
3.1 Digitale Signalverarbeitung 2
3.2 Die diskrete Fourier-Transformation 3
3.3 Faltung 4

4 Implementierung der diskreten Fourier-Transformation **4**

5 Eigenimplementierung eines Audioverarbeitungsprogramms **5**

6 Fazit **5**

Literaturverzeichnis **6**

1 Einleitung

In der heutigen Zeit, ermöglichen Programme wie Konferenzsoftware, Digitale Audio Workstations, Aufnahmeprogramme, Messagingprogramme und viele mehr, die Möglichkeit Töne, Sprache und Geräusche zu verarbeiten. Dies ist möglich dank digitaler Signalverarbeitung kombiniert mit Echtzeitlösungen wie Ringpuffern, Pufferung und vielem mehr. In diesem Dokument beleuchten wir einige bestehende Lösungen, welche die Audioverarbeitung auf Computern ermöglichen. Außerdem betrachten wir die Theorie der digitalen Signalverarbeitung und schauen uns auch die Implementierungen von diesen theoretischen Komzepten an. Darüber hinaus implementieren wir ein einfaches Audioverarbeitungsprogramm, welches eine Audiodatei annimmt und eine Filtrierung durch Fourier-Analyse vornimmt. Unter der Audiovearbeitung definieren wir die Komprimierung, Bearbeitung und Übertragung eines bestehenden Audiosignals.

1.1 Grundlagen und Begriffe

Audioverarbeitung Unter der Audioverarbeitung definieren wir die Komprimierung, Bearbeitung und Übertragung eines bestehenden AUdiosignals.

Audiobearbeitung Die Audiobearbeitung dient dazu, das Bearbeitungen an einem Audiosignal vorgenommen werden können. Einige Beispiele für solche Bearbeitungen sind Verzerrung, das Filtrieren von Frequenzbereichen (oder auch Frequenzbändern), das Anwenden eines Halleffektes oder auch das Verstellen bei der Audioaufnahme eines Musikstückes.

Pulsweitenmodulation

Abtrastrate

Bittiefe

Codec Codec ist die Kurzschreibweise für "Coder Decoder". Bei einem Codec wird das digitalisierte Audiosignal durch einen Kodierer gemäss eines gegebenen Formates kodiert (und bei komprimierten Codecs auch komprimiert). Diese kodierten und komprimierten Audiodaten kann man übertragen oder auch abspeichern und durch die Dekodierung dieses Signals, erhält man je nach Codec das Signal entweder in voller Gänze oder eine Annäherung davon (beispielsweise bei verlustbehafteter Audiokompression).

2 Bestehende Lösungen

Gemäß der Definition der Audioverarbeitung gibt es schon einige bestehende Lösungen für diese:

2.1 Bestehe Lösungen zur Audiokomprimierung

Die Audiokomprimierung dient dazu, dass ein digitalisiertes AUdiosignal in dessen Bandbreite zu komprimieren, um diese zur Übertragung, Absperichеrung oder Weiterverarbitung schneller und einfacher handzuhaben. Das analoge Audiosignal wird durch Pulsweitenmodulation abgetastet und digitaliert. Dabei haben die Abtastparameter, wie Abtrastrate und Bittiefe eine große Auswirkung auf der Bandbreite des digitalisierten Signals. Die Standardwerte für ein Musikstück beispielsweise sind eine Abtraste von 44.100 kHz und eine Bittiefe von 24 Bit. Hierbei bewegt man sich in den Größenbereichen von 20 Megabyte bis 60 Megabyte. Diese Größen sind nicht leicht handhabbar für die Übertragung oder gar beispielsweise für die Absperichеrung auf mobilen Endgeräte. Umegesetzt wird das ganze durch Codecs. Dabei muss man unterscheiden zwischen verlustfreien und verlustbehafteten Codecs.

Beispiele für verlustfreie Codecs sind **Free Lossless Audio Codec** (kurz: FLAC), **Wavpack**, **Apple Lossless Audio Codec** (kurz: ALAC) ,**Monkey Audio, True Audio** und weitere. [TZMK17]

Beispiele für verlustbehaftete Codecs hingegen sind **MPEG, Vorbis** oder auch **Windows Media Audio**.

Diese Lösungen werden je nach Anwendungsfall in heutiger Software, welche Audio benötigen weitesgehend verwendet.

2.2 Bestehende Lösungen zur Audiobearbeitung

Meist werden für die Audiobearbeitung digitale Audio Workstations verwendet. Einige Beispiele für solche digitalen Audio Workstations sind:

- Ableton Live
- FL Studio
- Bitwig Studio
- Apple Logic

3 Theorie

Damit die in 1.1 angegebenen Lösungen umgesetzt werden können, müssen entsprechende theoretische Grundlagen, dem Rechner vorgelegt werden. Im Falle der Audioverarbeitung basieren diese theoretischen Grundlagen auf der digitalen Signalverarbeitung.

3.1 Digitale Signalverarbeitung

Im Vergleich zu anderen Feldern in der Informatik, unterscheidet sich die digitale Signalverarbeitung in der Art der Daten, welche behandelt werden undzwar Signale. Diese Signale kommen meist von sensorischen Daten auf der echten Welt: Seien es

seismische Vibrationen, Bilder, Schwingungen durch Töne oder Geräusche etc. Die digitale Signalverarbeitung beinhaltet dabei die Mathematik, die Algorithmen und die Techniken, um digitalisierte Signale zu manipulieren. [Ste97]

In dieser Arbeit betrachten wir zwei Algorithmen aus der digitalen Signalverarbeitung. Zum einen die diskrete Fourier-Transformation und zum anderen die Faltung.

3.2 Die diskrete Fourier-Transformation

Mir der diskreten Fourier Transformation ist es möglich das Frequenzspektrum eines digitalisierten Signals zu berechnen. Dadurch erhält man die Informationen wie die Schwingungsfrequenz, Phase und Amplitude der einzelenen Teilschwingungen eines Gesamtsignals. [Ste97]

Dadurch erhält man neben der Zeit-Amplituden Repräsentation nun eine Amplituden-Frequnz Repräsentation des Signals, wodurch es möglich ist Bearbeitungen wie die Filtrierung von Frequenzbereichen eines Audiosignals vorzunehmen. Man spricht hierbei auch von einer Konvertierung von der Zeitdomöne zur Frequenzdomäne.

Bei der diskreten Fourier Transformation wird dabei wie folgt vorgegangen:

Gegeben sei ein kontinuierliches Signal dargestellt durch eine komplexe Funktion $f(t)$, wobei t die Zeit beschreibt. Dabei tasten wir dieses Signal ab und erhalten damit eine endliche Anzahl an $N \in \mathbb{N}$ Messpunkten in einem Abstand $T \in \mathbb{N}$. Diese notieren wir mit $f[0], f[1], \ldots, f[N-1]$.

Die Fourier-Transformation des ursprünglichen Signals $f(t)$ wäre dabei

$$F(j\omega) = \int_{-\infty}^{\infty} f(t)e^{-j\omega t}dt$$

Dadurch, dass wir aber mit einem diskreten Signal arbeiten, welches aus einzelenen Messpunkten bestehen, verändert sich unser diskretes Integral wie folgt:

$$\begin{aligned} F(j\omega) &= \int_{0}^{(N-1)T} f(t)e^{-j\omega t}dt \\ &= f[0]e^{j0} + f[1]e^{-j\omega T} + \cdots + f[N-1]e^{-j\omega(N-1)T} \end{aligned}$$

oder auch

$$F(j\omega) = \sum_{k=0}^{N-1} f[k]e^{-j\omega kT}$$

$F(j\omega)$ liefert uns dabei für eine Frequenz ω dessen Amplitudenanteil in dem Signal und j ist die imaginäre Zahl in $\mathbb{C}$

$$\omega \in \{0, \frac{2\pi}{NT}, \frac{2\pi}{NT} \cdot 2, \ldots, \frac{2\pi}{NT} \cdot (N-1)\}$$

Aus all dem ergibt sich die folgende Formel für die diskrete Fourier-Transformation:

$$F[n] = \sum_{k=0}^{N-1} f[k] e^{-j\frac{2\pi}{N}nk} \ (n = 0 : N-1)$$

3.3 Faltung

Bei der Faltung handelt es sich neben der diskreten Fourier-Transformation auch um eine Operation aus der digitalen Signalverarbeitung. Dabei werden zwei Signale f und g zusammengeführt. Mathematisch lässt sich diese Zusammenführung wie folgt beschreiben:

Sei h die Faltungsfunktion [GFV21]:

$$g(n) = \sum_k f(k)h(n-k) = \sum_k h(k)x(n-k)$$

Die Faltung findet sehr oft Verwendung in der Audiobearbeitung, um Halleffekte anhand eines gegeben Raumhallsignales zu realisieren. Diese Effekte nennt man auch entsprechend Faltungshall.

4 Implementierung der diskreten Fourier-Transformation

Dadurch, dass wir nun einige theoretische Grundlagen der Audioverarbeitung festgelegt haben, können wir diese nun implementieren. Dabei setzen wir den Fokus auf der diskreten Fourier-Transformation, da diese in der Audioverarbeitung am relevantesten ist und in vielerlei Audioverarbeitungssoftware Verwendung findet.

Der Pseduocode zur Implementierung der diskreten Fourier Transformation sieht wie folgt aus:

```
def discrete_fourier_transform(signal, samples):
    result = list()

    for k in range(0, N):
        sum = 0
        for n in range(0, N):
            angle = (-2 * PI * k * n) / N
        result.push(sum)
```

5 Eigenimplementierung eines Audioverarbeitungsprogramms

Nachdem wir mit der Implementierung der diskreten Fourier Transformation nun eine der wichtigsten Operationen der Audioverarbeitung implementiert haben, versuchen wir uns nun an der Implementierung eines vollwertigen Audioprogrammes. In der Audiosoftwareindustrie ist es dabei üblich, dass auf Programmiersprachen wie C, C++ oder auch Rust gesetzt werden. In dieser Arbeit findet die Implementierung in C++ statt, da diese am weitesten verbreitet ist zur Entwicklung von Audioverarbeitungsapplikationen mit der Begrüdung, dass hier auch durch direkter Ausführung auf der Hardware, die nötigen Resourcen zur Audioverarbeitung verwendet werde mit möglichst geringer Latenz.

Das Audioverarbeitungsprogramm was wir implementieren werden läuft auf der Kommandozeile und nimmt eine Audiodatei im RIFF-WAVE Format an. Über einen Kommandozeilenparameter wird die Frequenz angegeben, ab welcher eine Tiefenpassfiltrierung stattfinden soll, welche wir durch diskrete Fourier Transformation realisieren werden.

Das filtriete Signal soll dann als neue RIFF-WAVE Datei mit der Eingangsabtastrate und Bittiefe ausgegeben werden.

Die Syntax unseres Kommandozeilenprograms ist also wie folgt:

```
    simpleFilter -in <Eingabedatei> -freq <
       Filtrierungsfrequenz> -out <Ausgabedatei>
```

Dabei greifen wir auf die Bibliothek **AudioFile** von Adam Stark zurück, um das Dateihandling zu realisieren.

Die Implementierung ist verfügbar unter dem GitHub Repository:

https://github.com/anotherSimpleCoder/simpleFilter

6 Fazit

Auch wenn die Implementierung der einzelnen Audioverarbeitungsalgorithmen sich als sehr aufwändig feststellt, bilden diese die Grundlage der heutigen digitalen Audioverarbeitung, welche Internettelefonate, Musikproduktion und auch neue Arten des Umgangs mit Geräuschen und Klängen realisiert. Jedoch ist es heutzutage so, dass eine komplette Eigenimplementierung von diesen Operationen nicht mehr benötigt wird, da es für diese Bibliotheken wie **FFTW**, **KFR** oder auch **GNURadio** gibt.

Literaturverzeichnis

[GFV21] Jovanovic Dolecek Gordana and Alfonso Fernandez-Vazquez. Understanding discrete time convolution: A demo program approach. *Computación y Sistemas*, 25(2):361–367, 2021.

[Ste97] Smith Steven, W. *The Scientist and Engineer's Guide to Digital Signal Processing*. California Technical Pub, 1997.

[TZMK17] Surya Gunawan Teddy, M. Khalif Mat Zain, Fathiah Abdul Muin, and Mira Kartiwi. Investigation of lossless audio compre4ssion using ieee 1857.2 advanced audio coding. *Indonesian Journal of Electrical Engineering and Computer Science*, 6(2):422, 2017.